Resilient Branching MPC for Multi-Vehicle Traffic Scenarios Using Adversarial Disturbance Sequences

Victor Fors, Björn Olofsson, and Erik Frisk

***Abstract*—An approach to resilient planning and control of autonomous vehicles in multi-vehicle traffic scenarios is proposed. The proposed method is based on model predictive control (MPC), where alternative predictions of the surrounding traffic are determined automatically such that they are intentionally adversarial to the ego vehicle. This provides robustness against the inherent uncertainty in traffic predictions. To reduce conservatism, an assumption that other agents are of no ill intent is formalized. Simulation results from highway driving scenarios show that the proposed method in real-time negotiates traffic situations out of scope for a nominal MPC approach and performs favorably to state-of-the-art reinforcement-learning approaches without requiring prior training. The results also show that the proposed method performs effectively, with the ability to prune disturbance sequences with a lower risk for the ego vehicle.**

***Index Terms*—Autonomous driving, tactical decision making, uncertain systems, predictive control for nonlinear systems.**

I. Introduction

AUTONOMOUS vehicles have to make the same fast decisions as we humans make on the fly. Other road occupants could be expected to follow traffic rules and take logical actions, but there is no guarantee that they will meet these expectations. Reports from autonomous-driving experiments on public roads indicate that many accidents with other vehicles or incidences where a safety driver elects to take control to prevent a potential collision are caused by human error [1]. While challenging, the potential of autonomous driving in terms of increased safety is high [2].

Traffic prediction is an active research field encompassing a variety of methods [3], [4], methods that can be incorporated in a decision-making and planning framework. Even with good motion or intent predictions, the inherent uncertainty introduced by traffic is large compared to the typical uncertainty stemming from sensor measurements and modeling errors. With other agents on the road having similar capabilities as the ego vehicle, the potential deviations from the predicted behavior can have the same magnitude as the control-input limits. To improve robustness and resilience, and thereby to increase safety and reduce the number of accidents, the decision-making and planning of an autonomous vehicle should account for these large, often multi-modal, uncertainties arising from the surrounding traffic. While a number of methods in literature address this, they rely on prior data of the traffic behavior, which may not cover all possible critical events. This paper demonstrates a method that does not use such knowledge.

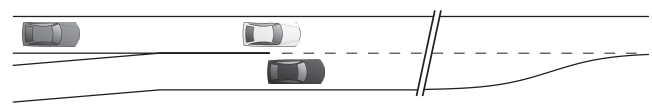

Fig. 1. Highway merging scenario with multiple vehicles interacting.

An interesting case study to highlight challenges for such a method is highway merging as illustrated in Fig. 1. In this scenario, the bottom vehicle is entering the highway and must merge before the entry lane ends. There is a trade-off between safety and comfort, because merging faster may be safer while keeping accelerations limited is more comfortable. The problem is non-convex from a decision point-of-view, with the possibility to merge in between or on either side of the other vehicles. Which side it is better to merge on can change depending on the actions of the other vehicles, meaning that the best merging maneuver is dependent on the uncertain behavior of these vehicles. One consequence of this is that a larger disturbance in the traffic is not necessarily harder to handle than a moderate disturbance, as previously occupied space can be freed up.

A. Related Research

With good knowledge of the environment, dynamic-programming methods such as lattice planning using A* can be used to efficiently plan a path and trajectory [5]. A weakness of the method comes from the computational requirements when the search space is large, which may require a rough discretization of the state space and a limited set of control-input segments. A flexible method for planning is Monte-Carlo tree search (MCTS) [6], which has been applied to the domain of motion planning for vehicles [7]. Similar to dynamic-programming methods, the computational requirements of Monte-Carlo tree

Manuscript received 17 January 2022; revised 4 March 2022 and 4 April 2022; accepted 8 April 2022. Date of publication 19 April 2022; date of current version 19 December 2022. This work was supported in part by Excellence Center at Linköping–Lund in Information Technology (ELLIIT) and in part by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. *(Corresponding author: Victor Fors.)*

Victor Fors and Erik Frisk are with the Division of Vehicular Systems, Department of Electrical Engineering, Linköping University, SE-581 83 Linköping, Sweden (e-mail: fors@stanford.edu; erik.frisk@liu.se).

Björn Olofsson is with the Division of Vehicular Systems, Department of Electrical Engineering, Linköping University, SE-581 83 Linköping, Sweden, and also with the Department of Automatic Control, Lund University, SE-221 00 Lund, Sweden (e-mail: bjorn.olofsson@liu.se).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TIV.2022.3168772.

Digital Object Identifier 10.1109/TIV.2022.3168772

2379-8858 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

search in a continuous domain such as the one of autonomous driving increase with a finer discretization of the action space.

Another approach is reinforcement-learning methods such as Deep Q-Network (DQN) [8], where a neural network learns a control policy from training on a large number of training episodes. Originally demonstrating its prowess in Atari games, it has also been applied to the domain of motion planning for vehicles [9]. Combining MCTS with neural network policies has found success in board games such as Go [10] and has also been applied to the domain of motion planning for vehicles [11]. The DQN method has been further developed for increased performance into methods such as Dueling DQN [12] and Quantile Regression DQN (QR-DQN) [13]. The DQN methods only work with discrete action spaces. Examples of reinforcement-learning methods that can handle continuous action spaces are Proximal Policy Optimization (PPO) [14] and Truncated Quantile Critics (TQC) [15]. A drawback with reinforcement-learning methods is that the learned policy can have difficulty in situations not covered by the training episodes.

A class of optimization methods that handle large state and action spaces well are the direct methods for optimal control [16], where the continuous-time state and input are approximated, e.g., using polynomials, and the resulting finite-dimensional optimization problem is solved using numerical methods. A disadvantage of these methods is that only a locally optimal solution is obtained, which poses a problem when considering non-convex problems. A way to mitigate this is by combining direct optimal control with a global optimization method such as lattice planning using A* [17]. An alternative method of mitigation is based on identifying collision-free driving corridors based on reachability analysis [18], [19] or convex optimization [20]. In Model Predictive Control (MPC), an open-loop optimization problem is typically solved using a receding horizon where only the first computed sample of the control input in the prediction horizon is applied to the system and the remaining samples are discarded, and then the process is repeated with an updated optimization problem for the new initial condition at the next sample [21].

A problem with computing an open-loop prediction is that uncertainties in the system model are not explicitly dealt with. There are many variations to the traditional MPC formulation (from now on referred to as nominal MPC) to handle these uncertainties. The robust open-loop MPC formulation [22] computes a control sequence that is feasible for all possible disturbances, often formulated as a minimax problem. This open-loop accounting of disturbances results in conservative solutions and can turn a feasible problem infeasible. Stochastic MPC [23] uses statistical distributions to model the uncertainty, transforming the robust hard constraints into chance constraints, allowing some probability of constraint violation. While the formulation enables less conservative planning, constraints have a certain probability to be violated, with less likely events being more likely to result in constraint violation. In tube MPC, the problem is reformulated as finding a sequence of control laws [24], [25]. A general feedback policy is prohibitively difficult to compute online, so the solution is approximated as a feedback policy that keeps the system in an invariant tube around the desired trajectory. Forming these invariant tubes poses a challenge for the domain of motion planning in multi-vehicle scenarios, because the desired driving corridor is dependent on the uncertain behavior of the other vehicles.

The notion of feedback present in the prediction can also be included as in feedback MPC [26], where each disturbance realization results in a branching of the predicted trajectory, forming a scenario tree of predicted inputs and states (further described in Section II). The resulting problem size grows exponentially with the prediction horizon N and is thus in practice only possible to solve for very short prediction horizons. Scenario MPC [27], [28] addresses this by sampling a limited number of disturbance realization from a statistical model of the uncertainty. The result is similar to stochastic MPC in that less likely events are less likely to be accounted for. On the opposite end, contingency MPC [29], [30] considers only an identified hazardous event, where the resulting alternative trajectory corresponding to the hazardous event occurring is selectively feasible to the identified hazard. A challenge lays in how to identify each such hazard.

B. Contributions

The main contribution is a motion planner and controller called Adversarial Disturbance-Sequence Branching MPC, denoted ADSB-MPC for short. In the domain of multi-vehicle traffic scenarios, ADSB-MPC accounts for uncertainty introduced by traffic and does this without relying on a statistical distribution of possible scenarios as in Scenario MPC, identified hazardous events as in Contingency MPC, or multiple training episodes in related scenarios as in reinforcement learning. The key technical contributions are: 1) a method to automatically identify a possible hazardous event resulting from the possible actions of another agent and to compute a corresponding disturbance sequence representing the deviation from the nominal predicted behavior, 2) an algorithm to generate a scenario tree with relevant disturbance sequences such that the resulting scenario tree grows linearly with both the prediction horizon and with the number of agents, and 3) an algorithm to prune the resulting scenario tree to obtain a fixed problem size with respect to the number of agents.

II. Principle of Planner

The proposed planner is based on MPC, which enables straightforward inclusion of constraints and traffic predictions over a finite-time horizon. To predict the future motion, a discrete model is considered that includes a bounded disturbance $\omega_k \in \mathbb{R}^{n_\omega}$ at each discrete time instant. The dynamics and constraints of the prediction model are described by the nonlinear discrete-time system

$$x_{k+1} = f(x_k, u_k, \omega_k) \tag{1a}$$

$$h_k(x_k, u_k, \omega_{0:k}) \leq 0 \tag{1b}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

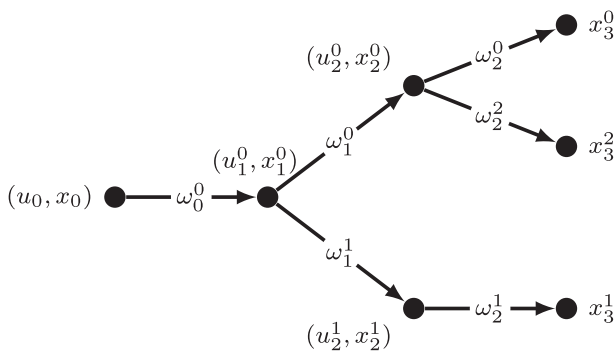

Fig. 2. Example of a scenario tree, e.g., arising in traffic with uncertainty. The nodes contain the different states x_k^j and control signals u_k^j at different time instants k and realizations j. A split in the tree occurs when there are more than one disturbance realization w_k^j originating from one node.

where $x_k \in \mathbb{R}^{n_x}$ denotes the state, $u_k \in \mathbb{R}^{n_u}$ denotes the input, $k = 0$ corresponds to the state at the current time instant, and the notation $\omega_{0:j} = \{\omega_0, \ldots, \omega_j\}$ defines that all future disturbances up to time instant j influence the constraint h_j at time instant j. Equations (1) describe the ego-vehicle motion and since the constraint function (1b) varies with time it can encode time-dependent constraints, e.g., moving obstacles.

To handle the disturbances ω_k, it would be ideal to have a control signal u_k for each possible disturbance sequence $\omega_{0:k}$. Then, as new measurements become available, the correct action can be taken with respect to the current disturbance realization. For a discrete set of disturbance realizations, each realization ω_k^j leads to an alternative state trajectory. The collection of alternative state trajectories can be represented by a scenario tree, an example of which is illustrated in Fig. 2. The point at which a branching occurs in the tree is the earliest point at which measurements of the actual disturbance realization are available for feedback. Specifically, after a disturbance realization ω_k^j occurred, the next state x_{k+1}^j and corresponding input u_{k+1}^j can be determined. The scenario tree is a representation of this presence of feedback in the prediction, which as in feedback MPC [26] is encoded in the optimal control problem (OCP) described later in Section V.

However, there are an infinite number of disturbance realizations and even if only the extreme values of ω_k are considered, the scenario tree grows exponentially with the prediction horizon N. Thus, not all possible disturbance sequences can be covered but there is still a benefit in considering some key sequences. This requires heavy pruning of the scenario tree with a careful selection of disturbance-sequence realizations and this is described in Section IV. In short, the determination of disturbance sequences to include takes advantage of the re-planning property of MPC by using the planned trajectory at the previous sample instant. The strategy is based on finding disturbance-sequence realizations that make the previously planned trajectory corresponding to the nominal prediction ($\omega_k = 0, \forall k$) infeasible. The selection is further narrowed down to at most a single disturbance-sequence realization per agent, by selecting the realization based on an adversarial objective function.

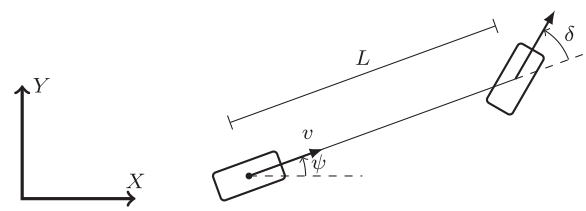

Fig. 3. Kinematic vehicle model.

III. Vehicle and Obstacle Modeling

In this section, the vehicle equations-of-motion are described and the approach to formulate obstacle constraints is introduced.

A. Kinematic Vehicle Model

The focus of the motion planning performed in the scenarios considered in this paper is on path and speed planning. For this purpose a kinematic vehicle model is used, which fulfills constraints imposed by the steering limit. Considering the vehicle to be operating under normal driving conditions with limited accelerations, the kinematic model is assumed to sufficiently represent the relevant aspects of the vehicle motion. To allow for more aggressive maneuvers, a more detailed vehicle model capturing, e.g., the tire–road interaction could be used in the same framework. The equations of motion for the kinematic vehicle model are [31]

$$\dot{X} = v\cos\psi, \qquad \dot{Y} = v\sin\psi, \qquad \dot{\psi} = v\frac{\tan\delta}{L} \tag{2a}$$

$$\dot{v} = u_a, \qquad \dot{\delta} = u_{\dot{\delta}} \tag{2b}$$

where (X, Y) is the position of the rear axle in an earth-fixed coordinate frame, ψ is the vehicle orientation in the same frame, v is the velocity, δ is the steering angle, L is the wheelbase, u_a is the acceleration input, and $u_{\dot{\delta}}$ is the steering-rate input. The model is illustrated in Fig. 3 as the kinematic single-track model, but it can also represent a traditional vehicle with Ackermann steering geometry. To obtain discrete-time dynamics of the form (1a), the equations of motion (2) are discretized using the Runge-Kutta fourth-order method RK4 [32].

B. Simplified Vehicle Model

For the purpose of fast lattice planning to compute disturbance sequences (see Section IV-C) and an initial guess for numerical optimization (see Section V-A), a simpler vehicle model is also used. This model is obtained by setting $\delta = \psi = 0$, where the lateral movement is instead determined by the lateral movement input $u_y = \dot{Y}/\dot{X}$. Written with X as the free variable, Y, v^2, and the time t as states, and u_a and u_y as constant inputs for each change ΔX of X, the model can be written in discrete time as:

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

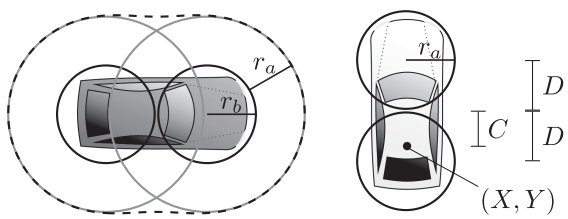

Fig. 4. Obstacle constraints with associated radii and distances involved in the modeling.

$$X_{i+1} = X_i + \Delta X \tag{3a}$$

$$Y_{i+1} = Y_i + u_y \Delta X \tag{3b}$$

$$v_{i+1}^2 = v_i^2 + 2u_a \Delta X \tag{3c}$$

$$t_{i+1} = t_i + \begin{cases} -\dfrac{v_i}{u_a} + \sqrt{\dfrac{v_i^2}{u_a^2} + \dfrac{2\Delta X}{u_a}}, & u_a > 0 \\ \dfrac{\Delta X}{v_i}, & u_a = 0. \\ -\dfrac{v_i}{u_a} - \sqrt{\dfrac{v_i^2}{u_a^2} + \dfrac{2\Delta X}{u_a}}, & u_a < 0 \end{cases} \tag{3d}$$

C. Obstacle Constraints

One circle is not suitable to represent the vehicle shape and orientation. Therefore, the obstacle constraints for the vehicles are formulated using a kernel representation based on two circles as seen in Fig. 4. This will ensure a smooth representation of the rectangular-shaped vehicle. The distance from the reference position (X, Y) at the rear axle to the center of the circles are determined by the distance C to the vehicle body center, and the distance D from the center of the vehicle body to the center of each circle. The positions of the center points (X_1, Y_1) and (X_2, Y_2) of the respective circle are determined by the relations

$$X_1 = X + (C + D)\cos(\psi) \tag{4a}$$

$$Y_1 = Y + (C + D)\sin(\psi) \tag{4b}$$

$$X_2 = X + (C - D)\cos(\psi) \tag{4c}$$

$$Y_2 = Y + (C - D)\sin(\psi). \tag{4d}$$

As illustrated by the two larger gray circles in Fig. 4, constraints between the center points a and b of two circles with the radii r_a and r_b, respectively, can be viewed as a larger circle constraining the point b a distance $r_a + r_b$ away from a and be formulated as

$$||a - b||_2^2 - (r_a + r_b - s_k)^2 \geq 0 \tag{5}$$

where s_k is introduced as a slack variable. For a more vehicle-shaped constraint than the two circles give, the constraints for both points on one vehicle are combined using the squared exponential kernel function with the length scale l. With points a_i, $i = 1, \ldots, N_i$, on one vehicle to compare to each point b_j, $j = 1, \ldots, N_j$, on another vehicle, the constraints are formulated as

$$\sum_{i=1}^{N_i} k(a_i, b_j) \leq 1, \forall j \tag{6a}$$

$$k(a, b) = \exp\left(-\frac{||a - b||_2^2 - (r_a + r_b - s_k)^2}{2l^2}\right). \tag{6b}$$

The resulting constraint for each circle center point a_1 and a_2 on the black vehicle in Fig. 4 is illustrated as a dashed black line overlaying the gray vehicle. For each target vehicle, the constraints are combined and compared to the center point of the circles overlaying the ego vehicle. The length scale l is selected such that the constraint boundary is at least the distance $r_a + r_b$ from the vehicle centerline:

$$l = \frac{D}{\sqrt{2 \ln 2}}. \tag{7}$$

The lane constraints are enforced as

$$Y_{j,k} - Y_{\min}(X_{j,k}) + s_k \geq r_j \tag{8a}$$

$$Y_{\max}(X_{j,k}) - Y_{j,k} + s_k \geq r_j, \forall j, k \tag{8b}$$

where $Y_{\min}$ and $Y_{\max}$ are functions of $X_{j,k}$ describing the lane limit along the Y-direction.

IV. Determining Disturbance Sequences

Even if it was computationally feasible to consider all possible disturbance sequences, it may not be desirable as the solution will be overly conservative. One way to deal with this is to introduce rules such as existing traffic laws, but other agents on the road would not necessarily follow the anticipated rules and traffic laws do not specify all interactions between traffic participants. There could be many traffic situations where there is a need for negotiation or to predict the risk of performing a maneuver to drive safely. A risk-averse strategy is here formulated by considering how other vehicles could disturb the ego vehicle if it drives according to its nominal behavior.

A. Building the Scenario Tree

To enable long prediction horizons, it is essential to only consider a carefully chosen subset of the possible disturbance realizations that influence the ego-vehicle solution. Even with feedback of the ego vehicle present in the predictions, an adversarial disturbance model can result in overly conservative solutions or even infeasible problems if worst-case behavior of surrounding traffic is considered. To reduce the conservatism compared to the worst-case adversarial disturbance model, it is assumed that other agents are not of ill intent, i.e., they will not negatively feedback on the motion of the ego vehicle to cause a collision but can still deviate significantly from their predicted behavior. This is formalized as an open-loop adversarial model of the disturbances.

Definition 1 (Open-loop adversarial): A disturbance-sequence realization $\omega_{0:N}$ is open-loop adversarial with respect to an initial state $\hat{x}_0$ and an input sequence $\hat{u}_{0:N}$, if it is bounded and $\exists\, j \in \{0, \ldots, N\}$ such that for (1), $h_j(x_j, u_j, \omega_{0:j}) >$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Generate Disturbance Tree $\mathcal{W}$.

Input: Initial state $\hat{x}_0$, Agents $\mathcal{A}$, Trajectory tree $\mathcal{T}$
Output: Disturbance tree $\mathcal{W}$
$\hat{u}_{0:N} \leftarrow \text{extract_nominal}(\mathcal{T})$
for each $a \in \mathcal{A}$ **do**
  $\omega_{k:N} \leftarrow \text{get_open_loop_adversarial}(a, \hat{x}_0, \hat{u}_{0:N})$
  if $\omega_{k:N} \neq \emptyset$ **then**
    Add $\omega_{k:N}$ to $\mathcal{W}$

Algorithm 2: Prune Disturbance Tree $\mathcal{W}$.

Input: Disturbance tree $\mathcal{W}_{\text{in}}$, Number of disturbance sequences n
Output: Disturbance tree $\mathcal{W}_{\text{out}}$
Sort each $\omega_{k:N}$ in $\mathcal{W}_{\text{in}}$ by (9)
$n \leftarrow \min(n, \text{number_of_sequences_in}(\mathcal{W}_{\text{in}}))$
Add the n first $\omega_{k:N}$ in $\mathcal{W}_{\text{in}}$ to $\mathcal{W}_{\text{out}}$

0, under the motion equations $x_{k+1} = f(x_k, u_k, \omega_k) \, \forall \, k \in \{0, \ldots, N\}$, $x_0 = \hat{x}_0$, and $u_{0:N} = \hat{u}_{0:N}$.

Out of the in total n_ω disturbances, each agent a has n_a disturbances $\omega^a \in \mathbb{R}^{n_a} \subseteq \mathbb{R}^{n_\omega}$ explicitly acting on it, e.g., deviations from the predicted actions of the agent such as unexpected acceleration or steering. Now, the objective is to formulate an algorithm that for each agent a collects a single open-loop adversarial sequence $\omega_{0:N}$ that explicitly acts on agent a. With N_a agents, the worst case computationally is that all N_a disturbance sequences obtained with such an algorithm start from the root node and the resulting scenario tree (see Fig. 2) contains $N + N_a(N-1)$ nodes. Note that this tree grows linearly with both the prediction horizon and with the number of agents and therefore the approach does not suffer from exponential complexity growth with increasing problem size.

The input to the algorithm is an estimation of the current state $\hat{x}_0$, the set of agents $\mathcal{A}$, and a trajectory tree $\mathcal{T}$ containing predicted inputs $\hat{u}$ obtained from the previous planning iteration by the MPC planner (see Section V). The output is a disturbance tree $\mathcal{W}$, describing how the disturbance sequences associated with each agent branch off the nominal disturbance sequence $\omega_{0:N} = 0$. The algorithm proceeds as follows: 1) from the previously planned trajectory tree $\mathcal{T}$, the nominal input trajectory $\hat{u}_{0:N}$ corresponding to no disturbance ($\omega_{0:N} = 0$) is extracted ($\text{extract_nominal}(\mathcal{T})$); 2) for each agent a in $\mathcal{A}$; 3) search for an open-loop adversarial disturbance sequence $\omega_{k:N}, k \in \{0, \ldots N\}$ that explicitly acts only on agent a, i.e., only the subspace of disturbances $\omega^a \in \mathbb{R}^{n_a} \subseteq \mathbb{R}^{n_\omega}$ in $\omega \in \mathbb{R}^{n_\omega}$ are nonzero. ($\text{get_open_loop_adversarial}(a, \hat{x}_0, \hat{u}_{0:N})$); and 4) if such a sequence exists; then 5) add it to the disturbance tree $\mathcal{W}$. The algorithm is summarized by Algorithm 1.

The complete scenario tree with states, inputs, and disturbances is formed by associating the resulting disturbance tree $\mathcal{W}$ and the nominal disturbance-free sequence $\omega_{0:N} = 0$ with the corresponding states and inputs as illustrated in Fig. 2.

B. Adversarial Objective Function

As stated, there are possibly infinitely many open-loop adversarial disturbance-sequence realizations, so a criterion is needed to determine which to consider. One important variable is the time, t_{inf}, indicating when the planned trajectory becomes infeasible. Before formal definitions, let $\mathcal{D}$ denote the set of all open-loop adversarial disturbance sequences and $t : \mathbb{N} \to \mathbb{R}$ be a map from a discrete-time index to the corresponding time.

Definition 2 (t_{inf}): For a disturbance $\omega_{0:N} \in \mathcal{D}$, then $t_{\text{inf}}(\omega_{0:N}) = t(k)$, where $k \in \{0, \ldots, N\}$ is the smallest k such that $\omega_{0:k} \in \mathcal{D}$.

It is reasonable to assume that a disturbance sequence that makes the planned trajectory infeasible earlier is more difficult to manage by re-planning. Similarly, a disturbance sequence that deviates from the nominal behavior later results in less time available to compensate for it. Therefore, another important variable is the starting time, t_{dist}, of the disturbance sequence.

Definition 3 (t_{dist}): For a disturbance $\omega_{0:N} \in \mathcal{D}$, then $t_{\text{dist}}(\omega_{0:N}) = t(k)$, where $k \in \{0, \ldots, N\}$ is the smallest k such that $||\omega_k|| \neq 0$.

Further, the magnitude of the resulting constraint violation and possibly the increase in objective cost could be considered. Thus, there is a trade-off between making the planned trajectory becoming infeasible early against a disturbance sequence starting later. Therefore, a trade-off parameter $\eta \in [0, 1)$ is introduced in a minimization problem with the adversarial objective function

$$\underset{\omega_{0:N}}{\text{minimize}} \, t_{\text{inf}}(\omega_{0:N}) - \eta \, t_{\text{dist}}(\omega_{0:N}), \; \omega_{0:N} \in \mathcal{D}. \tag{9}$$

At time t_{inf}, the future disturbance sequence is determined by a minimum-time solution that intentionally increases the resulting constraint violation (see Section IV-C).

By design, the disturbance tree $\mathcal{W}$ obtained using the algorithm in Section IV-A grows linearly with the number of agents. To obtain a fixed tree size with respect to the number of agents, the tree $\mathcal{W}$ can be pruned to only include the n most critical disturbance sequences. The adversarial objective function (9) can be used for this purpose, where the pruned tree is obtained by Algorithm 2.

C. Computing Disturbance Sequences

To compute the disturbance sequence that minimizes (9) for a specific target vehicle, a fast search on a discretized state lattice is performed using Dijkstra's algorithm [31]. Collisions with vehicles other than the ego vehicle are ignored, because this decouples the motion of each vehicle. Further, this choice allows to severely reduce the search space by not including the time t as a state on the lattice.

Using the simplified vehicle model (3), the lattice is formed on the variables X, v^2, and t_{dist} where the possible actions are $u_a = \pm a_{\text{dist}}$, or to keep on the nominal predicted trajectory. The lateral position Y_{TV} of the target vehicle is determined in a feedback fashion such that the target vehicle approaches the lateral position Y_{ego} of the ego vehicle by a fraction k_y of its

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

longitudinal movement

$$u_y = k_y \operatorname{sign}(Y_{\text{ego}} - Y_{\text{TV}}). \tag{10}$$

Infeasibility of the ego-vehicle trajectory is determined by the collision constraint (6) being unfulfilled without a slack s_k. Only open-loop adversarial disturbance-sequence realizations within the prediction horizon of the MPC are considered. When an adversarial disturbance-sequence realization is found, a minimum-time based control law is used to intentionally keep the constraint violation large for the remainder of the prediction horizon. The control law applies the acceleration on the target vehicle that does not reduce the velocity if it is below v_{min}, and will bring the target vehicle to the ego vehicle in minimum time if the ego vehicle keeps a constant velocity:

$$x = -\frac{|v_{\text{TV}} - v_{\text{ego}}|(v_{\text{TV}} - v_{\text{ego}})}{2a_{\text{dist}}} \tag{11a}$$

$$a_{\text{TV}} = \begin{cases} a_{\text{dist}}, & X_{\text{TV}} - X_{\text{ego}} < x \text{ and} \\ & v_{\text{TV}} + a_{\text{dist}}\Delta t^2/2 < v_{\text{min}} \\ -a_{\text{dist}}, & \text{otherwise} \end{cases} \tag{11b}$$

where $v_{\text{TV}} - v_{\text{ego}}$ is the difference in speed between the target vehicle and the ego vehicle.

D. Importance Weights

When formulating the motion planner objective function, see (14) in Section V, it is of interest to assign importance weights β to the different branches in the scenario tree. If the weights were to represent the true statistical distribution, then the objective function (14) is equivalent to minimizing the expected cost. Without knowing the true distribution, the challenge is how the weights should be assigned. The selection of weights are based on the following three principles:

Principle 1: The weights of all nodes corresponding to the same time instant sum up to 1.

Principle 2: Adding a branch retains the relative weights between existing branches.

Principle 3: The weights of outgoing branches from a node sum up to the incoming weight to the node.

Principle 2 means that the more branches that are included in the problem, the less likely each individual branch is. Principle 3 means that the weights remain consistent such that for any sub-tree of the scenario tree, the sum of the weights at each time instant is equal, which is what would be expected if interpreting the weights as probabilities. To fulfill Principles 1–3 and to enable some branches to be prioritized, the odds parameter γ is introduced to compute the importance weights of a scenario tree. The weight β_j for each child branch j from node i is computed as

$$\beta_j = \beta_i \frac{\gamma_j}{\sum_k \gamma_k} \tag{12}$$

where k is the number of outgoing branches from node i.

An example of a tree with assigned weights is illustrated in Fig. 5. To parameterize the tree, the nominal branch is given the odds parameter $\gamma_n = 1$ and the disturbance branches are given the odds parameter γ_d. The odds parameter γ_d should be considered a tuning parameter of how much, apart from being feasible, should the alternative branches influence the resulting optimization result. A large γ_d can make the problem more robust to disturbance-sequence realizations that are different, but similar, to those included in the scenario tree, while a smaller γ_d improves the performance under nominal conditions where the other agents behave as expected.

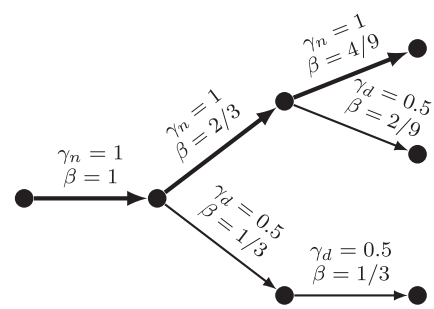

Fig. 5. Illustration of weights for each node in an example scenario tree. Each edge in the tree is parameterized with the odds parameter γ, resulting in the different importance weights β.

V. Motion Planning and Control

With the adversarial disturbance-sequence realization and the associated scenario tree computed according to Section IV, the motion planner and controller is defined as an MPC problem, resulting in the Adversarial Disturbance-Sequence Branching MPC (ADSB-MPC) strategy.

In the nominal case, where it is assumed that there are no disturbances, the MPC problem is formulated as

$$\underset{x,u,s}{\text{minimize}} \quad \sum_{k=1}^{N+1} \left(\begin{bmatrix} \Delta x_k \\ u_{k-1} \end{bmatrix}^{\text{T}} Q \begin{bmatrix} \Delta x_k \\ u_{k-1} \end{bmatrix} + b\, s_k \right) \tag{13a}$$

$$\text{subject to} \quad x_0 = \hat{x}_0 \tag{13b}$$

$$x_k = f(x_{k-1}, u_{k-1}), \; k = 1, \ldots, N+1 \tag{13c}$$

$$g(x_k, o_k, s_k) \leq 0, \; k = 1, \ldots, N+1 \tag{13d}$$

$$x_{\text{min}} \leq x_k \leq x_{\text{max}}, \; k = 1, \ldots, N+1 \tag{13e}$$

$$u_{\text{min}} \leq u_k \leq u_{\text{max}}, \; k = 0, \ldots, N \tag{13f}$$

$$s_k \geq 0, \qquad k = 1, \ldots, N+1 \tag{13g}$$

where the objective function (13a) is chosen as a quadratic stage cost where $\Delta x_k = x_k - x_{\text{ref}}$ is the state error relative to a low-complexity reference x_{ref} specifying the desired speed and lane, the matrix Q is positive semi-definite and b is a scalar, the constraint (13b) enforces that the prediction starts at the estimated current state $\hat{x}_0$, the constraints (13c) enforce the disturbance-free dynamics (2), the constraints (13d) enforce the disturbance-free obstacle constraints (6)–(8) with predicted obstacle positions o_k, the constraints (13e) enforce the state limits such as maximum speed, the constraints (13f) enforce the input limits such as maximum acceleration, and the constraints (13g) enforce that the slack variables s_k are non-negative.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

The corresponding branching optimal control problem that considers the adversarial disturbance-sequence realizations differs only in a few aspects to (13). To describe the scenario tree $\mathcal{T}$, illustrated in Fig. 2, the functions $p_x : \mathbb{R}^{n_x} \to \mathbb{R}^{n_x}$, and $p_u : \mathbb{R}^{n_x} \to \mathbb{R}^{n_u}$ are introduced. The function $p_x(x_k)$ returns the state of the parent node to state x_k and the function $p_u(x_k)$ returns the corresponding input that brings the system from $p_x(x_k)$ to x_k. For example, using the notation in Fig. 2, this gives $p_x(x_3^2) = x_2^0$ and $p_u(x_3^2) = u_2^0$. The total number of state-trajectory elements $1 + N_x$ is equal to $1 + \sum_b N_b$, where N_b is the length of each branch from its parent and the first term accounts for x_0. The total number of input-trajectory elements $1 + N_u$ is equal to $1 + \sum_b (N_b - 1)$ (the terminal state elements do not have corresponding inputs). Flattening the index notation such that $x_k,\ k = 0, \ldots, N_x$, are all the states in the scenario tree and $u_k,\ k = 0, \ldots, N_u$, are all the inputs, the ADSB-MPC optimal control problem is formulated as

$$\underset{x,u,s}{\text{minimize}} \quad \sum_{k=1}^{N_x} \beta_k \left(\begin{bmatrix} \Delta x_k \\ p_u(x_k) \end{bmatrix}^{\mathrm{T}} Q \begin{bmatrix} \Delta x_k \\ p_u(x_k) \end{bmatrix} + b\, s_k \right) \tag{14a}$$

$$\text{subject to} \quad x_0 = \hat{x}_0 \tag{14b}$$

$$x_k = f(p_x(x_k), p_u(x_k)),\ k = 1, \ldots, N_x \tag{14c}$$

$$g(x_k, o_k, s_k) \leq 0,\ \ k = 1, \ldots, N_x \tag{14d}$$

$$x_{\min} \leq x_k \leq x_{\max},\ k = 1, \ldots, N_x \tag{14e}$$

$$u_{\min} \leq u_k \leq u_{\max},\ k = 0, \ldots, N_u \tag{14f}$$

$$s_k \geq 0, \qquad k = 1, \ldots, N_x \tag{14g}$$

The importance weights β_k enter the objective function to prioritize minimizing the stage cost of branches with higher importance weight (see Section IV-D). The scenario tree is encoded in the formulation by the dynamic constraints (14c). Only disturbances affecting the obstacle positions o_k are considered and they are encoded in (14d) by different predicted positions o_k for each $k = 1, \ldots, N_x$. In general, disturbances affecting the dynamics (1a) can be encoded by different dynamic equations f_k for each $k = 1, \ldots, N_x$, and disturbances affecting the constraints (1b) can be encoded by different constraint equations h_k for each $k = 1, \ldots, N_x$.

The procedure for ADSB-MPC can be summarized as follows. At each sample, the input trajectory $\hat{u}_{0:N}$ corresponding to the disturbance-free branch is extracted from the solution to (14) at the previous sample and a new scenario tree with associated disturbance sequences is obtained according to the algorithm described in Section IV-A with importance weights computed according to Section IV-D. Each disturbance sequence building up the scenario tree is computed using the adversarial objective function described in Section IV-B and the procedure in Section IV-C. The optimization problem (14) is then re-solved with this new scenario tree and the control input obtained at the root of the resulting trajectory tree is applied to the system, and then this process is repeated at the next sample.

A. Initialization

Since the problems (13)–(14) are non-convex, the initialization of the variables in the optimal control problem is of high importance when considering numerical solution approaches. Solving (13) or (14) using a local optimization method means that the solver is not guaranteed to converge to a global optimum. To initialize the problem, it is therefore first approximated and solved using a global optimization method, in this case a search on a discretized state lattice using A* [31]. Similar to when finding adversarial disturbance sequences in Section IV, the simplified vehicle dynamics (3) are discretized on a lattice. Because the solution to (13) and (14) are dependent on moving obstacles and the lateral vehicle motion, the lattice is formed on the states X, Y, v^2, and t. While the time t does not conform to a lattice using (3), the search graph is reduced by forming cells with discretization Δt, allowing at most one vertex per cell for each unique X, Y, and v^2.

In total there are 5 actions available: keep constant velocity, accelerate with $a_{\max}$, brake with $-a_{\max}$, increase the lateral position, and decrease the lateral position. While the discretization of the input space is coarse to make the search real-time feasible, the trajectories are refined by solving (13) and (14) using direct optimal control, where the variables are initialized with the result from the lattice search. Compared to dynamic-programming approaches such as lattice search, direct optimal control scales well with respect to the prediction horizon and the discretization resolution of states and inputs.

To initialize the branching trajectory in (14), each disturbance sequence included in the problem is solved for individually using the lattice search. This means that in practice it is assumed that the global minimum does not significantly change when simultaneously optimizing for all branches, e.g., the optimal solution for a branch does not move from merging in front of a specific target vehicle to merging behind it.

VI. Results

Prototype implementations of the Nominal MPC (13) and the ADSB-MPC (14) strategies were made in Python and C++, using CasADi [33] to formulate the optimal control problems, which were subsequently solved numerically by the interior point optimizer Ipopt [34] together with the linear systems solver MA57 [35]. The ADSB-MPC is first evaluated in two hand-crafted merging scenarios and then in a large number of randomized highway-driving scenarios. A video illustrating the different simulations is available at https://youtu.be/etxvjGoX7RA.

Table I summarizes the parameters used in the evaluations, where w_{lane}, L, r, D, C, v_{ref}, $v_{\max}$, $v_{\min}$, $\psi_{\max}$, $\delta_{\max}$ and $a_{\max}$ are vehicle and scenario parameters. The reference x_{ref} is to drive straight in the center of the target lane with the speed v_{ref}. The values of Q and b in (13) and (14) are chosen such that the states $x_k = [X_k, Y_k, \psi_k, v_k, \delta_k]^{\mathrm{T}}$, the inputs $u_k = [u_{k,\dot{\delta}}, u_{k,a}]^{\mathrm{T}}$, and the slack s_k give the same contribution to the objective function for a deviation equal to their specified cost normalization (as per Table I):

$$Q = \operatorname{diag}(0, q_Y^{-2}, q_\psi^{-2}, q_v^{-2}, q_\delta^{-2}, q_{\dot{\delta}}^{-2}, q_a^{-2}), \quad b = q_s^{-1}. \tag{15}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETERS

Description	Notation	Scenario		Unit
		Merge	Highway	
Lane width	w_{lane}	3	4	m
Wheelbase	L	2	5	m
Collision circle radius	r	1.35	1.4	m
Collision circle separation	D	1.4	1.4	m
Vehicle body center	C	1	2.5	m
Maximum speed	$v_{\max}$	30	30	m/s
Minimum speed	$v_{\min}$	0	0	m/s
Maximum orientation	$\psi_{\max}$	$\pi/2$	$\pi/2$	rad
Maximum steering angle	$\delta_{\max}$	$\pi/6$	$\pi/4$	rad
Maximum acceleration	$a_{\max}$	3	5	m/s^2
Reference speed	v_{ref}	25	30	m/s
Lateral position cost	q_Y	3	12	m
Orientation cost	q_ψ	$\pi/2$	$\pi/2$	rad
Speed cost	q_v	10	20	m/s
Steering angle cost	q_δ	0.1	0.1	rad
Steering rate cost	$q_{\dot\delta}$	0.1	∞	rad/s
Acceleration cost	q_a	6	10	m/s^2
Slack cost	q_s	0.001	0.001	m
MPC prediction horizon	N	25	15	-
MPC sample time	-	0.2	0.2	s
Lattice position discretization	ΔX	5	5	m
Lattice time discretization	Δt	0.01	0.01	s
Branch odds	γ_d	0.5	0.5	-
Adversarial objective weight	η	0.25	0.25	-
Acceleration disturbance	a_{dist}	3	1	m/s^2
Lateral disturbance	k_y	0	0.1	-

The cost-normalization values are first selected based on worst case (similar to Bryson's rule [36]) and then q_δ, $q_{\dot\delta}$ were tuned to obtain smooth steering and q_v, q_a were tuned to obtain reasonable accelerations. In the highway scenario, the steering-rate cost $q_{\dot\delta}$ was set to ∞, since in that simulation the steering angle was directly used as the input. The parameters γ_d, η, a_{dist}, and k_y are specific to the ADSB-MPC. The values 0.25, 0.5, and 0.75 were tested for γ_d in the merging scenario, which gave similar results but different prioritization for the branches. The adversarial objective weight η is selected as a small value to prioritize smaller t_{inf}, but large enough such that for disturbance sequences with similarly sized t_{inf} the one with the largest t_{dist} is selected. In simulations, it has proven that the results are not very sensitive to the particular choice of η. The acceleration disturbance a_{dist} was lowered for the highway scenario to promote overtaking also in scenarios with dense traffic. The lateral disturbance parameter k_y was set to zero in the merging scenario, as the other agents were restricted to a single lane.

A. Merging

To make the merging scenario feasible, it is in the generation of disturbance sequences assumed that a target vehicle will not actively try to crash into the ego vehicle if the ego vehicle has merged into the same lane as the target vehicle, which is

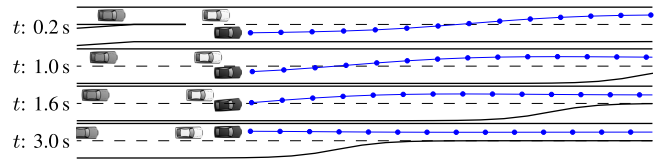

Fig. 6. Disturbance-Free Scenario: Black ego vehicle using the Nominal MPC strategy at different time instants. The future planned positions are illustrated with dots (with a sample period of 0.2 s).

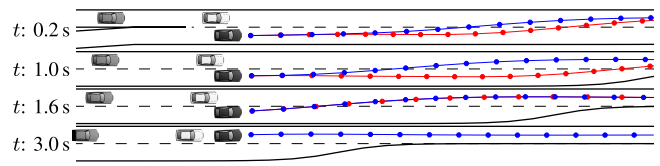

Fig. 7. Disturbance-Free Scenario: Black ego vehicle using the ADSB-MPC strategy at different time instants. The future planned positions are illustrated with dots (with a sample period of 0.2 s).

determined by the condition

$$|Y_{\text{TV}} - Y_{\text{ego}}| \leq \frac{w_{\text{lane}}}{2} \tag{16}$$

where Y_{TV} is the lateral position of the target vehicle, Y_{ego} is the lateral position of the ego vehicle, and w_{lane} is the lane width. Two different merging scenarios are examined: a disturbance-free scenario, where the other vehicles in the traffic are driving with constant velocity, and a disturbance scenario, where the white target vehicle performs a 3 m/s^2 acceleration, followed by a deceleration phase.

1) Disturbance-Free Scenario: In the disturbance-free merging scenario, the target vehicles behave as the nominal prediction in the MPC controllers, i.e., drive with a constant velocity. As seen in Figs. 6 and 7, both the Nominal MPC and the proposed ADSB-MPC are able to successfully perform a merging maneuver in this scenario. At early stages of the scenario, the ADSB-MPC has an alternative branched trajectory where, if the white car were to accelerate, the black ego vehicle would merge behind the white vehicle, which is seen in the alternative lower plans in the top illustrations in Fig. 7. Later, once the ego vehicle has high enough relative velocity and is far enough in front of the white vehicle, the alternative plan is to simply accelerate more to avoid collision in case the white vehicle performs an acceleration. Throughout the maneuver, there is no disturbance sequence affecting the gray vehicle that would prevent the black ego vehicle from merging.

2) Disturbance Scenario: In the disturbance scenario, the other traffic participants do not behave as predicted. The Nominal MPC fails to perform a merging maneuver and the ego vehicle ends up in a crash with another vehicle as seen in Fig. 8. The ADSB-MPC strategy, however, can handle also this scenario, as can be observed in Fig. 9. Like in the disturbance-free scenario, the ADSB-MPC in the disturbance scenario has an alternative plan to merge behind the white vehicle, if the white vehicle were to accelerate. Because of this, the ADSB-MPC does not accelerate as heavily as the resulting control action

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

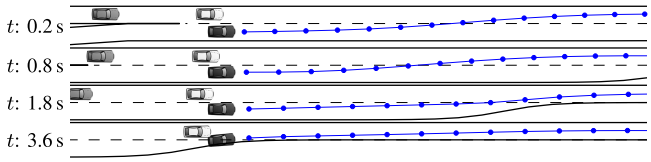

Fig. 8. Disturbance Scenario: Black ego vehicle using the Nominal MPC planner at different time instants. The future planned positions are illustrated with dots (with a sample period of 0.2 s).

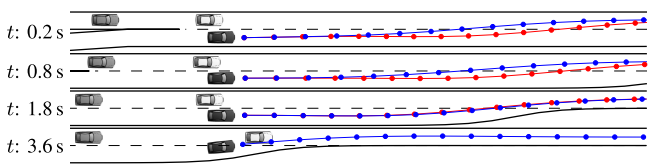

Fig. 9. Disturbance Scenario: Black ego vehicle using the ADSB-MPC planner at different time instants. The future planned positions are illustrated with dots (with a sample period of 0.2 s).

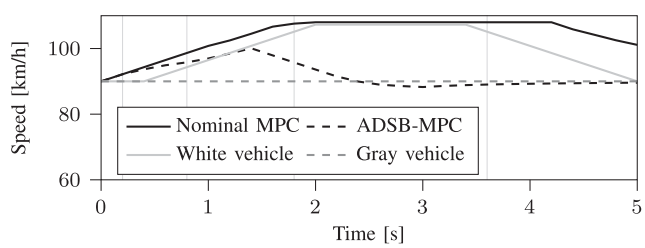

Fig. 10. Disturbance Scenario: Speed. The vertical light gray lines correspond to the time instants illustrated in Figs. 8 and 9.

from the Nominal MPC (see Fig. 10). By 1.8 s, where the Nominal MPC had reached an infeasible optimization problem, the ADSB-MPC is driving sufficiently slower than the white vehicle to plan a safe trajectory to merge behind it, and the ego vehicle eventually merges safely (see Fig. 9).

B. Highway Driving

To examine the controller performance in a tactical decision-making task, the ADSB-MPC is implemented for a highway driving scenario in the simulation environment Highway-Env [37] aimed at evaluating reinforcement-learning agents. This enables comparison with reinforcement-learning methods and will test the ability of ADSB-MPC to handle a large number of randomized scenarios with, for the controller, unknown traffic behavior where multiple surrounding vehicles drive at varying velocities and overtake each other. A number of reinforcement-learning agents were trained using Dueling DQN [12], QR-DQN [13], PPO [14], and TQC [15], of which TQC performed the best and is used here to compare with the ADSB-MPC planner. A capture of a highway driving scenario in Highway-Env is shown in Fig. 11. The behavior of the traffic in Highway-Env is simulated using the Intelligent Driver Model (IDM) [38] to determine the accelerations and the Minimizing Overall Braking Induced by Lane change (MOBIL) model [39] to determine lane changes.

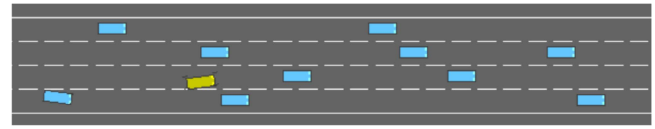

Fig. 11. Capture from Highway-Env during execution of the ADSB-MPC planner. The ego vehicle is shown as yellow.

The goal of the ego vehicle is to drive fast and keep to the rightmost lane without crashing into the other vehicles. Specifically, the reward function used to evaluate the scenario and train the TQC agent is a linear combination of the reward obtained for crashing r_{crashed}, the reward obtained for being closer to the correct lane r_{lane}, and the reward for keeping the velocity high r_{speed}:

$$r = \frac{b_c(1 - r_{\text{crashed}}) + b_l r_{\text{lane}} + b_v r_{\text{speed}}}{b_c + b_l + b_v} \tag{17}$$

where $b_c = 1$, $b_l = 0.1$, and $b_v = 0.4$ are parameters. The crash reward r_{crashed} is 1 if the ego vehicle has crashed and 0 otherwise. The lanes are numbered in increasing order from 0 to 3 from the leftmost lane to the rightmost lane. The lane reward is determined by the current lane $i_{\text{lane}} \in \{0, 1, 2, 3\}$ and is given by

$$r_{\text{lane}} = \frac{i_{\text{lane}}}{n_{\text{lanes}} - 1} \tag{18}$$

where $n_{\text{lanes}} = 4$ is the total number of lanes. The speed reward r_{speed} is given by a linearly mapped function between $v_{r,\min} = 20$ m/s and $v_{r,\max} = 30$ m/s:

$$r_{\text{speed}} = \begin{cases} 0, & v < v_{r,\min} \\ \frac{v - v_{r,\min}}{v_{r,\max} - v_{r,\min}}, & v_{r,\min} \le v \le v_{r,\max} \\ 1, & v > v_{r,\max} \end{cases}. \tag{19}$$

The developed ADSB-MPC is compared to the nominal MPC and TQC in 100 episodes of 20 s each, for three different traffic densities with all planners running at 5 Hz. Here, the pruning method proposed in Section IV-B was used to only include the $n = 2$ most critical open-loop disturbance sequences in the optimization problem for the ADSB-MPC. The TQC agent was trained using the implementation from Stable-Baselines3 [40]. First, the TQC agent was trained for 5,000,000 steps in the default traffic density d, then the TQC agent was trained for another 1,000,000 steps for each step of increasingly denser traffic, first $1.5d$ and then $2d$, resulting in three different TQC agents, TQC d, TQC $1.5d$, and TQC $2d$, specialized at driving in different traffic densities. Note that the objective of the comparison is not to find the best performing reinforcement-learning approach, but to compare state-of-the-art approaches to the proposed ADSB-MPC and discuss their properties.

Table II shows the success rate as the number of episodes out of a 100 without a collision and the accumulated reward as a percentage of the maximum possible reward. At low traffic density, success rate is high for both MPC and TQC but TQC has a higher reward. It is important to note that the MPC planners do not have the reward function (17) used in the evaluation as an

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUCCESS RATES AND ACCUMULATED REWARDS IN HIGHWAY-ENV

Traffic density	Planner	Success	Reward
Low / d	TQC d	100/100	99 %
	Nominal MPC	98/100	95 %
	ADSB-MPC	100/100	93 %
Medium / $1.5d$	TQC $1.5d$	90/100	93 %
	Nominal MPC	90/100	89 %
	ADSB-MPC	99/100	89 %
High / $2d$	TQC $2d$	63/100	76%
	Nominal MPC	66/100	70 %
	ADSB-MPC	85/100	77 %

objective function, so this is not unexpected. When increasing the traffic density by 50%, the reward of the dedicated TQC planner drops closer to that of the MPC planners, with a significantly lower success rate than ADSB-MPC. Increasing the traffic density to double that of the low traffic density, the ADSB-MPC outperforms the other planners, including the TQC agent trained on high-density data. In summary, the ADSB-MPC is the planner that is least likely to crash, which comes at the cost of more careful driving than with the Nominal MPC.

VII. DISCUSSION

A key aspect of ADSB-MPC is that the considered disturbance sequences are neither handcrafted nor is a statistical model of the disturbances needed. Instead, they are automatically determined as part of the ADSB-MPC strategy, taking the capabilities of the other agents in the scenario into account. In the results in Section VI, the ADSB-MPC only used the simple prediction model that assumes the traffic to keep a constant velocity. ADSB-MPC performs well in the randomized highway scenarios (see Table II) where the traffic is realistically modeled using IDM and MOBIL, indicating the applicability of the method on real-world driving. As demonstrated in the merging scenario (see Fig. 9), unknown critical scenarios caused by other agents could be handled by ADSB-MPC if the possible actions of the agents are sufficiently modeled. Being able to handle dangerous situations not previously encountered and that the system is not explicitly designed for is a promising property of ADSB-MPC.

Here, we studied a motion planner and controller with moderate control authority. For emergency maneuvers, where the problem is identified as infeasible with this level of control authority, a system with a higher control authority could be straightforwardly used. Further, the method could be extended with a contingency scenario to, e.g., guarantee that it is possible to stop the ego vehicle before the current lane ends.

The ADSB-MPC approach results in an optimization-problem size that scales linearly with respect to the number of vehicles and the prediction horizon, which is appealing from a computational point-of-view. ADSB-MPC also enables the problem size to be fixed with respect to the number of vehicles by using the adversarial objective function (9) to only include the n most critical disturbance sequences in the problem. This is promising with regard to the scalability of the method when considering scenarios with many target vehicles.

Concerning the computational complexity, the procedure for determining the disturbance sequences is for the prototype implementation negligible compared to solving the optimal control problem in the MPC. On average, approximately 5% of the computational time is spent by the lattice planner to compute the disturbance sequences and the initial guess, 30% of the time is spent in Ipopt to solve the optimization problem, 45% of the time is spent computing expressions for derivatives to feed Ipopt before each sample, and the remainder is overhead spent in CasADi to formulate the optimal control problem and initialize Ipopt. With few vehicles on the road, such as in the merging scenario, the prototype implementation runs in real-time on an i7-7700HQ CPU.

VIII. CONCLUSION

It is important that future driver-assistance systems and autonomous vehicles can handle hazardous events not encountered during their development, ADSB-MPC addresses the methodological development in this direction. The proposed ADSB-MPC strategy uses automatically determined disturbance sequences that are intentionally adversarial to the ego vehicle. To reduce conservatism, the concept of open-loop adversarial is used. This leads to a resilient planning and control method, where the problem size scales linearly with the number of agents and the prediction horizon. Further, the ADSB-MPC strategy can prune the problem to only include the most critical disturbance sequences to obtain a fixed problem size with respect to the number of agents. The simulation results for multi-vehicle scenarios demonstrate that the ADSB-MPC can handle unknown critical scenarios without prior data of the potential hazard. Further, ADSB-MPC performs favorably to a TQC agent in high-density traffic without requiring prior training to obtain its policy and can negotiate traffic situations out of scope for a nominal MPC approach.

REFERENCES

[1] M. Schwall, T. Daniel, T. Victor, F. Favaro, and H. Hohnhold, "Waymo public road safety performance data," 2020, *arXiv:2011.00038*.
[2] B. Olofsson and L. Nielsen, "Using crash databases to predict effectiveness of new autonomous vehicle maneuvers for lane-departure injury reduction," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3479–3490, Jun. 2021.
[3] S. Lefèvre, D. Vasquez, and C. Laugier, "A survey on motion prediction and risk assessment for intelligent vehicles," *Robomech J.*, vol. 1, no. 1, pp. 1–14, 2014.
[4] S. Mozaffari, O. Y. Al-Jarrah, M. Dianati, P. Jennings, and A. Mouzakitis, "Deep learning-based vehicle behavior prediction for autonomous driving applications: A review," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 1, pp. 33–47, Jan. 2022.
[5] O. Ljungqvist, N. Evestedt, D. Axehill, M. Cirillo, and H. Pettersson, "A path planning and path-following control framework for a general 2-trailer with a car-like tractor," *J. Field Robot.*, vol. 36, no. 8, pp. 1345–1377, 2019.
[6] C. B. Browne *et al.*, "A survey of Monte Carlo tree search methods," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 1, pp. 1–43, Mar. 2012.
[7] D. Lenz, T. Kessler, and A. Knoll, "Tactical cooperative planning for autonomous highway driving using Monte-Carlo tree search," in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2016, pp. 447–453.
[8] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–33, 2015.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.

[9] C.-J. Hoel, K. Wolff, and L. Laine, "Automated speed and lane change decision making using deep reinforcement learning," in *Proc. 21st Int. Conf. Intell. Transp. Syst.*, 2018, pp. 2148–2155.
[10] D. Silver *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.
[11] C.-J. Hoel, K. Driggs-Campbell, K. Wolff, L. Laine, and M. J. Kochenderfer, "Combining planning and deep reinforcement learning in tactical decision making for autonomous driving," *IEEE Trans. Intell. Veh.*, vol. 5, no. 2, pp. 294–305, Jun. 2020.
[12] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
[13] W. Dabney, M. Rowland, M. Bellemare, and R. Munos, "Distributional reinforcement learning with quantile regression," in *Proc. 32nd AAAI Conf. AI*, 2018, pp. 2892–2901.
[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[15] A. Kuznetsov, P. Shvechikov, A. Grishin, and D. Vetrov, "Controlling overestimation bias with truncated mixture of continuous distributional quantile critics," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 5556–5566.
[16] M. Diehl, H. Bock, H. Diedam, and P.-B. Wieber, *Fast Direct Multiple Shooting Algorithms for Optimal Robot Control*, vol. 340. Berlin, Germany: Springer, 2006, pp. 65–93.
[17] K. Bergman, O. Ljungqvist, and D. Axehill, "Improved path planning by tightly combining lattice-based path planning and optimal control," *IEEE Trans. Intell. Veh.*, vol. 6, no. 1, pp. 57–66, Mar. 2021.
[18] S. Manzinger, C. Pek, and M. Althoff, "Using reachable sets for trajectory planning of automated vehicles," *IEEE Trans. Intell. Veh.*, vol. 6, no. 2, pp. 232–248, Jun. 2021.
[19] L. Schäfer, S. Manzinger, and M. Althoff, "Computation of solution spaces for optimization-based trajectory planning," *IEEE Trans. Intell. Veh.*, to be published, doi: 10.1109/TIV.2021.3077702.
[20] M. Morsali, E. Frisk, and J. Åslund, "Spatio-temporal planning in multi-vehicle scenarios for autonomous vehicle using support vector machines," *IEEE Trans. Intell. Veh.*, vol. 6, no. 4, pp. 611–621, Dec. 2021.
[21] M. Morari and J. H. Lee, "Model predictive control: Past, present and future," *Comput. Chem. Eng.*, vol. 23, no. 4, pp. 667–682, 1999.
[22] M. V. Kothare, V. Balakrishnan, and M. Morari, "Robust constrained model predictive control using linear matrix inequalities," *Automatica*, vol. 32, no. 10, pp. 1361–1379, 1996.
[23] T. Brüdigam, M. Olbrich, D. Wollherr, and M. Leibold, "Stochastic model predictive control with a safety guarantee for automated driving," *IEEE Trans. Intell. Veh.*, to be published, doi: 10.1109/TIV.2021.3074645.
[24] W. Langson, I. Chryssochoos, S. Raković, and D. Mayne, "Robust model predictive control using tubes," *Automatica*, vol. 40, no. 1, pp. 125–133, 2004.
[25] B. T. Lopez, J.-J. E. Slotine, and J. P. How, "Dynamic tube MPC for nonlinear systems," in *Proc. Amer. Control Conf.*, 2019, pp. 1655–1662.
[26] P. Scokaert and D. Mayne, "Min-max feedback model predictive control for constrained linear systems," *IEEE Trans. Autom. Control*, vol. 43, no. 8, pp. 1136–1142, Aug. 1998.
[27] G. Cesari, G. Schildbach, A. Carvalho, and F. Borrelli, "Scenario model predictive control for lane change assistance and autonomous driving on highways," *IEEE Intell. Transp. Syst. Mag.*, vol. 9, no. 3, pp. 23–35, 2017.
[28] I. Batkovic, U. Rosolia, M. Zanon, and P. Falcone, "A robust scenario MPC approach for uncertain multi-modal obstacles," *IEEE Control Syst. Lett.*, vol. 5, no. 3, pp. 947–952, Jul. 2021.
[29] J. Hardy and M. Campbell, "Contingency planning over probabilistic obstacle predictions for autonomous road vehicles," *IEEE Trans. Robot.*, vol. 29, no. 4, pp. 913–929, Aug. 2013.
[30] J. P. Alsterda and J. C. Gerdes, "Contingency model predictive control for linear time-varying systems," 2021, *arXiv:2102.12045*.
[31] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.
[32] U. M. Ascher and L. R. Petzold, *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. Philadelphia, PA, USA: SIAM, 1998.
[33] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi - A software framework for nonlinear optimization and optimal control," *Math. Program. Comput.*, vol. 11, no. 1, pp. 1–36, 2019.
[34] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Math. Prog.*, vol. 106, no. 1, pp. 25–57, 2006.
[35] HSL, "A collection of Fortran codes for large scale scientific computation," Accessed: Sep. 08, 2021. [Online]. Available: http://www.hsl.rl.ac.uk
[36] G. F. Franklin, J. D. Powell, A. Emami-Naeini, and J. D. Powell, *Feedback Control of Dynamic Systems*, 6th ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2009.
[37] E. Leurent, "An environment for autonomous driving decision-making," Accessed: Feb. 18, 2022. [Online]. Available: https://github.com/eleurent/highway-env
[38] M. Treiber, A. Hennecke, and D. Helbing, "Congested traffic states in empirical observations and microscopic simulations," *Phys. Rev. E, Statist. Phys., Plasmas, Fluids, Relat. Interdiscip. Top.*, vol. 62, pp. 1805–24, 2000.
[39] A. Kesting, M. Treiber, and D. Helbing, "General lane-changing model MOBIL for car-following models," *Transp. Res. Rec.*, vol. 1999, no. 1, pp. 86–94, 2007.
[40] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

Victor Fors received the M.Sc. degree in engineering physics from Lund University, Lund, Sweden, in 2015 and the Ph.D. degree in electrical engineering from Linköping University, Linköping, Sweden, in 2020. He is currently a Postdoc with Stanford University, Stanford, CA, USA. His main research interests include vehicle dynamics, control, active safety, and autonomy.

Björn Olofsson received the M.Sc. degree in engineering physics and the Ph.D. degree in automatic control from Lund University, Lund, Sweden, in 2010 and 2015, respectively. He is currently a Researcher with the Division of Vehicular Systems, Linköping University, Linköping, Sweden, and with the Department of Automatic Control, Lund University. His research interests include motion control for robots and vehicles, optimal control, system identification, and statistical sensor fusion.

Erik Frisk was born in Stockholm, Sweden, in 1971. He received the Ph.D. degree in 2001. He is currently a Professor with the Department of Electrical Engineering, Linkoping University, Linkoping, Sweden. His main research interests include optimization techniques for autonomous vehicles in complex traffic scenarios and model and data-driven fault diagnostics and prognostics.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 28,2026 at 10:54:18 UTC from IEEE Xplore. Restrictions apply.